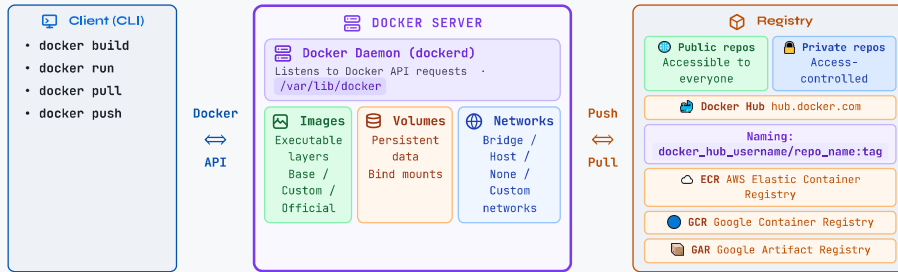




What is Docker? Docker allows developers to package applications along with their libraries & dependencies into a standardised unit called a container. Containers are platform-independent — they run the same on any machine. They allow applications to run in resource-isolated processes, eliminating infrastructure dependency issues ("works on my machine" problems). Unlike VMs, containers are lightweight and share the host OS kernel.

DOCKER ARCHITECTURE

CLIENT — SERVER ARCHITECTURE



SETUP

<code>sudo usermod -aG docker ubuntu</code>	Add user to docker group
<code>sudo systemctl enable docker</code>	Start docker on boot
<code>docker -d</code>	Start docker daemon
<code>docker info</code>	Kernel, images, containers info
<code>docker version --format</code>	Version details
<code>systemctl restart docker</code>	Restart docker service

COMPLETE IMAGE & CONTAINER FLOW



DOCKER RUN — KEY FLAGS

EXAMPLES

<code>docker run -it -p host_port:container_port jenkins/jenkins:lts</code>	Run Jenkins example creating a container from builtin image, goto web browser ip/localhost:9090
<code>docker run -d -p 9090:80 apache-centos:v1</code>	
<code>docker run -d -p host_port:container_port nginx:latest</code>	Nginx in background
<code>docker run -v myvol2:/app nginx:latest</code>	Volume mount
<code>docker run -p 8081-8085:9091-9095 img</code>	Port range (host:container)

FLAGS

<code>-d detached</code>	<code>-it interactive terminal</code>	<code>-dti detached + interactive</code>
<code>-p host:container port mapping</code>	<code>-P publish all exposed ports</code>	
<code>-v NAME:PATH volume mount</code>	<code>--mount src,tgt explicit mount</code>	
<code>--network NET attach network</code>	<code>--name NAME container name</code>	<code>--rm auto-remove on stop</code>
<code>-m 512m memory limit</code>	<code>--cpus="1.5" CPU limit</code>	<code>--pids-limit 100 max processes</code>
<code>--restart POLICY restart policy</code>	<code>-e VAR=VAL env variable</code>	

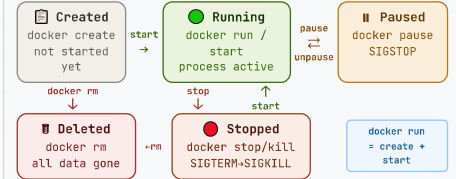
CREATE / START

<code>docker run -dti image_name:tag</code>	Create + start detached and interactive terminal
<code>docker run --rm image_name:tag</code>	Auto-remove on stop
<code>docker start cont_id</code>	Start stopped container
<code>docker restart cont_id</code>	Restart container

STOP / PAUSE

<code>docker stop cont_id</code>	Graceful stop
<code>docker stop -t 60 cont_id</code>	Stop, 60s timeout
<code>docker pause cont_id</code>	Pause (freeze)
<code>docker unpause cont_id</code>	Unpause (resume)
<code>docker kill cont_id</code>	Force kill (SIGKILL)

CONTAINER LIFECYCLE



INSPECT / LOG

<code>docker container ls</code>	List containers
<code>docker ps</code> <code>docker ps -a</code> <code>docker ps -q</code>	Running containers ALL containers IDs only
<code>docker ps -f status=exited</code>	Exited only
<code>docker inspect cont_id</code>	Full config JSON
<code>docker logs cont_id</code>	Container logs

DELETE / POLICY

<code>docker rm cont_id</code>	Remove stopped container
<code>docker rm \$(docker ps -q -f status=exited)</code>	Remove all exited
<code>docker ps -f status=exited xargs -l {} docker rm</code>	Remove exited (alt)
<code>docker restart cont_id</code>	Restart container
<code>docker system prune -a</code>	Remove unused images/containers/networks
<code>docker network prune</code> <code>docker image prune</code> <code>docker volume prune</code>	Remove unused networks Remove unused images Remove unused volumes
<code>docker update --restart unless-stopped \$(docker ps -q)</code>	

ENTER / EXEC CONTAINER

<code>docker exec -it cont_id /bin/bash</code>	New bash shell exit closes bash only, not container
<code>docker attach cont_id</code>	Attach to main process exit closes the container!
<code>Ctrl+Q+P</code>	Safe exit (detach)

CGROUPS (RESOURCE CONTROL)

<code>-m 512m</code> <code>--memory 512m</code>	Max 512 MB RAM (OOM Killer)
<code>--cpus="1.5"</code>	Max 1.5 CPU cores
<code>--pids-limit 100</code>	Max 100 processes

NAMESPACES (Isolation):

PID NET UTS IPC USER

QUICK REFERENCE

<code>docker run</code>	create + start
<code>docker start</code>	start stopped container
<code>docker exec</code>	run in existing container
<code>docker attach</code>	connect to main process



DOCKERFILE INSTRUCTIONS



What is Dockerfile?

Plain text file with commands to build an image. Each command creates a layer. All layers together = one image.

```
FROM ubuntu:22.04
(PYTHON:3.5/php:3.5/CENTOS:7/UBUNTU:latest)
RUN apt-get update
COPY ./app /app
ADD file.tar.gz # also downloads URL + auto-extracts
/app
WORKDIR /app
ENV PORT=8080 ENV=prod
ARG BUILD_VER=1.0
EXPOSE 8080
CMD ["node", "app.js"]
ENTRYPOINT ["/run.sh"]
MAINTAINER MAHESH
```

if src is an URL, ADD will automatically download and extracts (if its zipped) to target path.

ARG	Build-time only	ENV	Build + runtime
CMD	Can be overridden	ENTRYPOINT	Fixed entry (use --entrypoint to override at runtime)

CMD vs ENTRYPOINT: CMD = default execution (can be overridden). ENTRYPOINT = entry point (more stable).

BEST PRACTICES:

- ✓ One service per container
- ✓ Build context as small as possible
- ✓ Avoid unnecessary packages
- ✓ Minimize number of layers

IMAGE TYPES

Official	Base from Docker Hub
Base	Minimal OS layer
Custom	Your Dockerfile

BUILD FLAGS

docker build -t image_name:tag . /path	Build & tag from current dir
-t name:tag	Tag the image
-f Dockerfile.alt	Use alternate Dockerfile
--build-arg K=V	Pass ARG at build time

IMAGE COMMANDS

docker images	List all images -a option
docker image ls	List images (same as above)
docker images -f dangling=true	Dangling (untagged) images
docker pull image_name:tag	Pull from registry
docker search NAME	Search Docker Hub
docker rmi IMAGE	Remove image
docker rmi -f IMAGE	Force remove
docker rmi \$(docker images -f dangling=true -q)	Remove all dangling
docker image prune -a --filter "until=12h"	Prune old images
docker commit cont_ID image_name:tag	Snapshot from container/creating image from container

Adding Dockerfile instruction while importing the Image using -c

```
docker -c Dockerfile_Instruction archive_file image_name
Example: docker -c 'CMD echo "HelloWorld!!!"'
helloworld.tar my-imported-img:v4
```

IMAGE FILTERS & PRUNE

-f dangling=true	Untagged images
-f status=exited	Exited containers
--filter "until=12h"	Older than 12 hours
--filter "label=unused"	By label
--force / -f	Force remove
--no-prune	Keep untagged parents
docker image prune -a --filter "until=12h"	Prune old images
docker images -a grep none awk '{print \$3}' xargs docker rmi --force	Remove <none> images

REGISTRY - PUSH / PULL

docker login docker login -u USER -p PASS	Authenticate to Docker Hub Login with credentials
docker tag local_image:tag hub_USER/repo_name:tag Format: hub_username/repo_name:tag	Tag image for Hub
docker push USER/repo:tag	Push to Hub
docker pull USER/repo:tag	Pull from Hub
preferred name: user_name/repo:tag	

NETWORKS

Purpose: To customize IP, to communicate between containers

bridge	- default network, - Removes isolation between host and docker, - containers communicate via bridge IP, - Use bridge driver by default if not specified
host	- Container shares host's networking stack directly, - Uses host driver, - container shows host IP address
none	- No networking - Fully isolated container - for security, network troubleshooting, sensitive workloads, compliance requirements, network failure testing, high security CI/CD jobs
custom	User-defined bridge, shown under bridge driver

Host Default IP: 172.17.0.0/16

docker network ls	List networks (id, name, driver, scope)
docker network create network_name	Create custom network
docker network create --driver bridge network_name	Create bridge network
docker network inspect bridge	Network details + container IPs
docker network connect network_name container_id	Connect container to network
docker network disconnect network_name container_id	Disconnect container from network
docker network rm NET	Remove network
ip a	List host networks

PORT MAPPING / EXPOSE

HOST Port :8080	CONTAINER Port :80 EXPOSE 80
-p 8080:80	Host:Container port map
-p 8081-8085:9091-9095	Port range (must match)
-P	Publish all EXPOSED ports
EXPOSE allows container-to-container only. Not accessible outside. PUBLISH (-p) = EXPOSE + publish. Accessible to outside world.	

RUN ON SPECIFIC NETWORK `docker run -dti --network NET IMAGE:TAG` `docker run -d -it --network host IMAGE:TAG`

SAVE / LOAD / EXPORT

docker save name.tar container_name	Save container to tar
docker export container_name filesystem	Export container filesystem
docker load -i image_name.tar	Load image from tar
docker import archive_image_name	Import from archive

MONITORING IN PRODUCTION

docker logs container_ID /var/lib/docker/container_id/-json.log	
docker stats container_id (or) docker stats volume_name	CPU, memory, network usage
docker events	attach, commit, die, rename...
docker events -f	Live event stream
docker ps -q xargs -L docker logs	Logs for all containers

MULTI-STAGE BUILD

```
FROM golang:1.21 AS builder
WORKDIR /app
COPY . .
RUN go build -o main
FROM alpine:latest
--from=builder /app/main /
COPY ["main"]
```

Multiple FROM statements separate build & runtime environments.
Combined with AS alias + --from=alias to copy only required artifacts.
Result: Smaller, cleaner, more secure final image.



VOLUMES & BIND MOUNTS

VOLUMES

- Managed by Docker
- `/var/lib/docker/volumes`
- Preferred for data
- easy to manage using cli
- easy to backup
- safely shared among multiple containers
- if you dont specify volumewhile creating container, annonomous volume will be created

BIND MOUNTS

- Managed by host
- Any host path
- Great for dev
- direct access to host filesystem
- less portable

VOLUME TYPES:

Named Volumes: Managed by Docker, stored in host location
Anonymous Volumes: Automatically created if not specified when creating container
Host Volumes (Bind Mounts): Direct mount from host filesystem
Benefits: Easy CLI management, backup-friendly, safely shared among containers

VOLUMES COMMANDS

<code>docker volume create VOL</code>	Create volume
<code>docker volume ls</code>	List volumes
<code>docker volume inspect VOL</code>	Volume details
<code>docker volume rm VOL</code>	Remove volume
<code>docker volume prune</code>	Remove unused volumes
Ex: <code>docker run -v myvol2:/app nginx</code>	-v attach volume to container
<code>--mount source=myvol2,target=/app</code>	Explicit mount syntax

Run container with volume (Or) Attach volume to container
`docker run -v volume_name:container_path image`
Ex: `docker run -v myvol2:/app nginx`

Explicit mount syntax with `--mount`
`source=myvol2,target=/app`
`docker run -d --name devtest \`
`--mount source=myvol2,target=/app \`
`nginx:latest`

start a service with volume
`docker service create -d \`
`--replicas=1 \`
`--name devtest-service \`
`--mount source=myvol2,target=/app \`
`nginx:latest`

REMOVE CONTAINER & VOLUME

Remove a volume

```
docker container stop nginx(cont_name)
docker container rm nginx(cont_name)
docker volume rm nginx_vol
```

Remove a container (always stop the container before removing)

```
docker container ls -l --filter=nginx --format '{{.Name}}'
docker stop $(cat /dev/null)
docker rm $(cat /dev/null)
```

Remove all containers

```
docker container rm $(docker container ls -aq)
docker container stop $(docker container ls -aq) &&
docker system prune -af --volumes
```

DRAIN USE CASES

- Service update without downtime
- Rolling updates for high availability
- Node maintenance (drain = evacuate tasks from node)

DOCKER COMPOSE (DOCKER-COMPOSE.YML)

To run/start/stop/delete multiple containers/services at once.

YAML STRUCTURE

```
version: '3.8'
services:
  web:
    image: nginx:latest
    container_name: myapp
    ports:
      - "8080:80"
    networks:
      - mynet
    environment:
      - ENV=prod
    volumes:
      - myvol:/data
    depends_on:
      - db
  db:
    image: postgres:15
volumes:
  myvol:
networks:
  mynet:
```

`depends_on` - starts container only when dependency is up
COMPOSE COMMANDS

<code>docker compose up -d</code>	Start all services (detached)
<code>docker compose -f name.json up</code>	Use alternate file
<code>docker compose ps</code>	List compose containers
<code>docker compose logs</code>	View logs
<code>docker compose pause</code>	Pause (keep state)
<code>docker compose unpause</code>	Unpause
<code>docker compose stop</code>	Stop (data preserved)
<code>docker compose down</code>	Remove containers, networks, volumes

COMPOSE LIFECYCLE



KEY CONCEPTS & INTERVIEW POINTS

Q Can a container restart by itself?

No by default, Yes - using restart policies:
`off` `on-failure` `always` `unless-stopped`

Q docker info shows what?

Kernel version, total images, running containers, storage driver, runtime info

Q Difference: EXPOSE vs PUBLISH (-p)?

`EXPOSE` = container-to-container only. `-p` = publish to outside world. Publish = expose + publish.

Q ARG vs ENV?

`ARG` available build-time only. `ENV` available at both build & runtime.

Q docker attach vs docker exec?

`attach` connects to main process (exit closes container). `exec` creates new process (exit only closes session).

Q How does Docker achieve isolation?

Namespaces (PID, NET, UTS, IPC, USER) + **cgroups** (CPU, Memory, Disk I/O, Network bandwidth, Processes)

Q Volumes vs Bind Mounts?

Volumes = managed by Docker (`/var/lib/docker/volumes`). Bind mounts = managed by host, any path.

FULL COMMAND QUICK REF

CONTAINER

`ls` `start` `stop` `restart` `pause`
`unpause` `rm` `inspect` `logs` `stats`

IMAGE

`pull` `push` `build` `tag` `rmi` `images`
`commit` `prune` `save` `load`

VOLUME

`create` `ls` `inspect` `rm` `prune`

NETWORK

`create` `ls` `inspect` `connect`
`disconnect` `rm`

IMPORTANT PATHS

<code>/var/lib/docker/volumes</code>	Docker volumes
<code>/var/lib/docker/containers/cont_id</code>	Container data
<code>/var/lib/docker/containers/cont_id/-json.log</code>	Container log file

COMMON PATTERNS & TIPS

Remove all exited containers:

```
docker rm $(docker ps -q -f status=exited)
```

Auto-restart all currently running:

```
docker update --restart unless-stopped $(docker ps -q)
```

Remove all <none> dangling images:

```
docker images -a | grep none | awk '{print $3}' | xargs docker rmi -f
```

Safe exit from container (detach):

`Ctrl + Q + P` - detaches without stopping the container

View live Docker events:

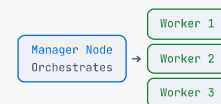
```
docker events -f -- attach, commit, die, rename, destroy...
```

YAML = Yet Another Markup Language

Used by docker-compose.yml for multi-container service definitions

DOCKER SWARM

Group of physical/virtual machines running Docker in cluster mode.





DOCKER VS VM VS HYPERVISOR VS CONTAINER RUNTIME

DOCKER

containers Virtualize the Software layers above the OS. Uses a container runtime / execution engine on the host. Shares the host OS kernel, no separate OS per container Packages applications along with their dependencies. Lightweight and fast startup(boot time). More portable and resource-friendly. Depends on the host operating system kernel. Best foe micro services and CICD pipelines.

CONTAINER RUNTIME

A container runtime manages the lifecycle of containers.

Responsibilities

- Create, start, stop, and delete containers.
- Provide isolation using namespaces and cgroups.
- Share the host OS kernel.

Examples:containerd,CRI-O



VIRTUAL MACHINE (VM)

VM virtualize the entire machine including Hardware, Storage, Network, CPU, operating system

Uses a hypervisor to run a full guest operating system.

Each VM has:

- Its own OS,Its own kernel,Virtualized hardware
- Provides stronger isolation.
- Higher resource and storage overhead.
- Slower boot time compared to containers.

HYPERVISOR (VMM - VIRTUAL MACHINE MONITOR)

A hypervisor creates and manages virtual machines.

Responsibilities

- Allocates CPU, memory, storage, and networking to VMs.
- Provides virtual hardware to guest operating systems.
- Enables multiple VMs to run on the same physical machine.

Types of Hypervisors

Type 1 Hypervisor (Bare-Metal)

- Runs directly on physical hardware.
- Does not require a host OS.
- Can directly access hardware and networking.
- Better performance and security.

Examples

VMware ESXi
Microsoft Hyper-V
Xen

Type 2 Hypervisor (Hosted)

- Runs on top of an existing host operating system.
- Uses the host OS for hardware access.
- Easier for desktop/lab usage.

Examples

Oracle VirtualBox,VMware Workstation



SAMPLE DOCKERFILE PROGRAMS

Program 1 – Ubuntu + Apache

```
FROM ubuntu
RUN apt-get update && \
    apt-get install -y apache2
apache2-utils && \
    apt-get clean
EXPOSE 80
CMD ["apache2ctl", "-D", "FOREGROUND"]
```

Program 2 – CentOS + httpd

```
FROM centos
RUN yum -y install httpd
EXPOSE 80
CMD ["apachectl", "-D", "FOREGROUND"]
```

Program 3 – Python 2.7 app

```
FROM python:2.7
RUN mkdir -p /app
WORKDIR /app
COPY requirements.txt /app/
RUN pip install -r requirements.txt
COPY . /app
CMD ["python", "main.py"]
```

Program 4 – Ubuntu 18.04 app

```
FROM ubuntu:18.04
COPY . /myapp
RUN make /myapp
CMD ["python", "/myapp/app.py"]
```

Program 5 – Node 9.4 app

```
FROM node:9.4
RUN mkdir -p /app
WORKDIR /app
COPY package.json /app/
RUN npm install
COPY . /app
ENTRYPOINT ["npm"]
CMD ["start"]
```

Program 6 – Tomcat with WAR

```
FROM ubuntu
RUN apt-get -y update && \
    apt-get -y install wget openjdk-8-jdk zip unzip
RUN mkdir -p /tomcat
WORKDIR /tomcat
RUN wget https://dlcdn.apache.org/tomcat/tomcat-10/v10.0.27/
bin/apache-tomcat-10.0.27.zip && \
    unzip apache-tomcat-10.0.27.zip
COPY target/SimpleWebApplication.war /tomcat/
apache-tomcat-10.0.27/webapps/
RUN chown -R root:root /tomcat && \
    chmod -R +x apache-tomcat-10.0.27/bin
EXPOSE 8080
CMD ["/tomcat/apache-tomcat-10.0.27/bin/catalina.sh", "run"]
```

Note: .war file should exist in the build context before copying.

Program 7 – Multistage build sample – Java app

```
FROM maven:3.6-jdk-8-alpine AS builder
WORKDIR /app
COPY pom.xml .
RUN mvn -e -B dependency:resolve
COPY src ./src
RUN mvn -e -B package
FROM openjdk:8-jre-alpine
COPY --from=builder /app/target/app.jar /app.jar
CMD ["java", "-jar", "/app.jar"]
```

Program 8 – Multistage build sample – Node + Nginx

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build
FROM nginx:alpine
COPY --from=build /app/build /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Program 9 – Multistage build sample – Node + Nginx

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/build /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

et

Program 10 – Compose sample – SonarQube + Postgres

```
version: "3"
services:
  sonarqube:
    image: sonarqube:lts
    container_name: sonarqube_new
    ports:
      - 9000:9000
    networks:
      - sonarnet
    environment:
      - SONARQUBE_JDBC_URL=jdbc:postgresql://db:5432/sonar
      - SONARQUBE_JDBC_USERNAME=sonar
      - SONARQUBE_JDBC_PASSWORD=sonar
    volumes:
      - sonarqube_conf:/opt/sonarqube/conf
      - sonarqube_data:/opt/sonarqube/data
      - sonarqube_extensions:/opt/sonarqube/extensions
      - sonarqube_bundled-plugins:/opt/sonarqube/lib/bundled-plugins

  db:
    image: postgres:11.5
    container_name: postgres
    ports:
      - 5432:5432
    networks:
      - sonarnet
    environment:
      - POSTGRES_USER=sonar
      - POSTGRES_PASSWORD=sonar
    volumes:
      - postgresql:/var/lib/postgresql
      - postgresql_data:/var/lib/postgresql/data

networks:
  sonarnet:

volumes:
  sonarqube_conf:
  sonarqube_data:
  sonarqube_extensions:
  sonarqube_bundled-plugins:
  postgresql:
  postgresql_data:
```